클라이언트 SDK 사용법 및 예제

개요

Irys 체인 클라이언트 SDK는 Rust로 작성된 강력한 라이브러리로, Irys 블록체인과의 상호작용을 위한 완전한 기능을 제공합니다. 이 SDK를 사용하여 트랜잭션 생성, 서명, 전송, 데이터 업로드, 청크 관리 등의 작업을 수행할 수 있습니다.

설치 및 설정

Cargo.toml 의존성 추가

```
[dependencies]

irys-api-client = "0.1.0"

irys-types = "0.1.0"

tokio = { version = "1.0", features = ["full"] }

eyre = "0.6"

serde_json = "1.0"
```

기본 설정

```
use irys_api_client::{ApiClient, ApiClientExt, IrysApiClient};

use irys_types::{

ConsensusConfig, IrysSigner, DataTransaction, DataTransactionHeader,
```

```
CommitmentTransaction, CommitmentType, DataLedger, H256, U256
};
use std::net::SocketAddr;


#[tokio::main]
async fn main() -> eyre::Result<()> {
// API 클라이언트 초기화
let client = IrysApiClient::new();
// 노드 주소 설정
let node_addr: SocketAddr = "127.0.0.1:8081".parse()?;
// 설정 초기화
let config = ConsensusConfig::testing();
// 서명자 생성
let signer = IrysSigner::random_signer(&config);
Ok(())
}
```

핵심 구성 요소

1. API 클라이언트 (ApiClient)

API 클라이언트는 Irys 노드와의 HTTP 통신을 담당합니다.

```
use irys_api_client::{ApiClient, IrysApiClient};
```

```rust
// 클라이언트 생성
let client = IrysApiClient::new();

// 커스텀 타임아웃으로 클라이언트 생성
let client = IrysApiClient {
client: reqwest::Client::builder()
.timeout(std::time::Duration::from_secs(10))
.build()?,
};
```

2. 서명자 (IrysSigner)

서명자는 트랜잭션 생성 및 서명을 담당합니다.

```rust
use irys_types::{IrysSigner, ConsensusConfig};
use k256::ecdsa::SigningKey;

// 랜덤 서명자 생성
let config = ConsensusConfig::testing();
let signer = IrysSigner::random_signer(&config);

// 기존 개인키로 서명자 생성
let private_key = SigningKey::from_slice(&hex::decode("your_private_key_hex")?)
```

```
let signer = IrysSigner {
signer: private_key,
chain_id: config.chain_id,
chunk_size: config.chunk_size,
};

// 서명자 주소 확인
println!("Signer address: {:?}", signer.address());
```

데이터 트랜잭션 워크플로우

1. 데이터 저장 가격 조회

```
use irys_api_client::ApiClientExt;
use irys_types::DataLedger;

async fn get_storage_price(
client: &IrysApiClient,
node_addr: SocketAddr,
data_size: u64,
) -> eyre::Result<()> {
// Publish 원장의 가격 조회
let price_info = client
```

```
.get_data_price(node_addr, DataLedger::Publish, data_size)
.await?;
println!("Storage price for {} bytes:", data_size);
println!(" Permanent fee: {}", price_info.perm_fee);
println!(" Term fee: {}", price_info.term_fee);
println!(" Ledger: {}", price_info.ledger);
Ok(())
}
```

2. 트랜잭션 생성 및 서명

```
async fn create_and_sign_transaction(
signer: &IrysSigner,
data: Vec<u8>,
anchor: Option<H256>,
) -> eyre::Result<DataTransaction> {
// 기본 트랜잭션 생성
let transaction = signer.create_transaction(data.clone(), anchor)?;
// 또는 수수료를 명시적으로 설정하여 생성
let transaction = signer.create_transaction_with_fees(
data,
anchor,
DataLedger::Publish,
```

```
U256::from(1000), // term_fee
Some(U256::from(10000)), // perm_fee
)?;
// 트랜잭션 서명
let signed_transaction = signer.sign_transaction(transaction)?;
// 서명 검증
assert!(signed_transaction.header.is_signature_valid());
Ok(signed_transaction)
}
```

3. 트랜잭션 제출

```
async fn submit_transaction(
client: &IrysApiClient,
node_addr: SocketAddr,
transaction: &DataTransaction,
) -> eyre::Result<()> {
// 트랜잭션 헤더 제출
client
.post_transaction(node_addr, transaction.header.clone())
.await?;
println!("Transaction submitted: {:?}", transaction.header.id);
Ok(())
```

```
}
```

4. 청크 업로드

```
async fn upload_chunks(
client: &IrysApiClient,
node_addr: SocketAddr,
transaction: &DataTransaction,
) -> eyre::Result<()> {
// 모든 청크 업로드
client.upload_chunks(node_addr, transaction).await?;
println!("All chunks uploaded for transaction: {:?}", transaction.header.id);
Ok(())
}

// 스트리밍 방식으로 청크 업로드
async fn upload_chunks_streaming(
client: &IrysApiClient,
node_addr: SocketAddr,
transaction: &DataTransaction,
data_iter: impl Iterator<Item = eyre::Result<Vec<u8>>> + Send,
) -> eyre::Result<()> {
client
```

```
.upload_chunks_iter(node_addr, transaction, data_iter)
.await?;
Ok(())
}
```

5. 트랜잭션 상태 확인

```
async fn check_transaction_status(
client: &IrysApiClient,
node_addr: SocketAddr,
tx_id: H256,
) -> eyre::Result<()> {
// 트랜잭션 정보 조회
let tx_response = client.get_transaction(node_addr, tx_id).await?;
println!("Transaction: {:?}", tx_response);
// 프로모션 대기
client.wait_for_promotion(node_addr, tx_id, 50).await?;
println!("Transaction promoted: {:?}", tx_id);
Ok(())
}
```

커밋먼트 트랜잭션

1. 스테이킹

```
async fn stake_tokens(
signer: &IrysSigner,
client: &IrysApiClient,
node_addr: SocketAddr,
config: &ConsensusConfig,
) -> eyre::Result<()> {
// 앵커 조회
let anchor = client.get_anchor(node_addr).await?;
// 스테이크 트랜잭션 생성
let stake_tx = CommitmentTransaction::new_stake(config, anchor);
// 서명
let signed_stake_tx = signer.sign_commitment(stake_tx)?;
// 제출 (커밋먼트 트랜잭션 제출 API 사용)
// Note: 실제 구현에서는 commitment_tx 엔드포인트 사용
println!("Stake transaction created: {:?}", signed_stake_tx.id);
Ok(())
}
```

2. 플레지

```
use irys_types::transaction::PledgeDataProvider;
```

```rust
// 플레지 데이터 제공자 구현
struct MockPledgeProvider {
pledge_count: u64,
}

#[async_trait::async_trait]
impl PledgeDataProvider for MockPledgeProvider {
async fn pledge_count(&self, _user_address: Address) -> u64 {
self.pledge_count
}
}

async fn pledge_tokens(
signer: &IrysSigner,
config: &ConsensusConfig,
anchor: H256,
) -> eyre::Result<()> {
let provider = MockPledgeProvider { pledge_count: 0 };
// 플레지 트랜잭션 생성
let pledge_tx = CommitmentTransaction::new_pledge(
config,
anchor,
```

```
&provider,
signer.address(),
).await;
// 서명
let signed_pledge_tx = signer.sign_commitment(pledge_tx)?;
println!("Pledge transaction created: {:?}", signed_pledge_tx.id);
println!("Pledge value: {}", signed_pledge_tx.value);
Ok(())
}
```

데이터 검색

1. 청크 조회

```
async fn retrieve_chunk(
client: &IrysApiClient,
node_addr: SocketAddr,
ledger: DataLedger,
data_root: H256,
offset: u32,
) -> eyre::Result<()> {
let chunk = client
.get_chunk(node_addr, ledger, data_root, offset)
```

```
.await?;
match chunk {
ChunkFormat::Packed(packed_chunk) => {
println!("Retrieved packed chunk with {} bytes", packed_chunk.data.len());
}
ChunkFormat::Unpacked(unpacked_chunk) => {
println!("Retrieved unpacked chunk: {:?}", unpacked_chunk.data_root);
}
}
Ok(())
}
```

2. 블록 정보 조회

```
use irys_api_server::routes::block::BlockParam;

async fn get_block_info(
client: &IrysApiClient,
node_addr: SocketAddr,
) -> eyre::Result<()> {
// 최신 블록 조회
let latest_block = client
.get_block_by_param(node_addr, BlockParam::Latest)
```

```
.await?;
println!("Latest block height: {}", latest_block.irys.height);
println!("Block hash: {:?}", latest_block.irys.block_hash);
// 특정 높이의 블록 조회
let block_at_height = client
.get_block_by_param(node_addr, BlockParam::BlockHeight(100))
.await?;
println!("Block at height 100: {:?}", block_at_height.irys.block_hash);
Ok(())
}
```

완전한 예제: 파일 업로드

```
use std::fs;
use irys_api_client::{ApiClient, ApiClientExt, IrysApiClient};
use irys_types::{ConsensusConfig, IrysSigner, DataLedger};

async fn upload_file_example(file_path: &str) -> eyre::Result<()> {
// 설정 및 초기화
let client = IrysApiClient::new();
let node_addr: SocketAddr = "127.0.0.1:8081".parse()?;
let config = ConsensusConfig::testing();
let signer = IrysSigner::random_signer(&config);
```

```
// 파일 읽기
let  file_data = fs::read(file_path)?;
let  file_size = file_data.len() as u64;
println!("Uploading file: {} ({} bytes)", file_path, file_size);
// 1. 가격 조회
let  price_info = client
.get_data_price(node_addr, DataLedger::Publish, file_size)
.await?;
println!("Storage cost:");
println!(" Permanent fee: {}", price_info.perm_fee);
println!(" Term fee: {}", price_info.term_fee);
// 2. 앵커 조회
let  anchor = client.get_anchor(node_addr).await?;
// 3. 트랜잭션 생성
let  transaction = signer.create_publish_transaction(
file_data,
Some(anchor),
price_info.perm_fee,
price_info.term_fee,
)?;
// 4. 트랜잭션 서명
let  signed_transaction = signer.sign_transaction(transaction)?;
println!("Transaction ID: {:?}", signed_transaction.header.id);
```

```
println!("Data root: {:?}", signed_transaction.header.data_root);
println!("Chunks: {}", signed_transaction.chunks.len());
// 5. 트랜잭션 제출
client
.post_transaction(node_addr, signed_transaction.header.clone())
.await?;
println!("Transaction submitted successfully");
// 6. 청크 업로드
client
.upload_chunks(node_addr, &signed_transaction)
.await?;
println!("All chunks uploaded successfully");
// 7. 프로모션 대기
client
.wait_for_promotion(node_addr, signed_transaction.header.id, 100)
.await?;
println!("Transaction promoted successfully");
// 8. 최종 상태 확인
let  final_tx = client
.get_transaction(node_addr, signed_transaction.header.id)
.await?;
println!("Final transaction status: {:?}", final_tx);
Ok(())
```

```
}
```

고급 기능

1. 배치 트랜잭션 처리

```
async fn batch_upload(
client: &IrysApiClient,
node_addr: SocketAddr,
signer: &IrysSigner,
files: Vec<Vec<u8>>,
) -> eyre::Result<Vec<H256>> {
let mut tx_ids = Vec::new();
// 앵커 한 번만 조회
let anchor = client.get_anchor(node_addr).await?;
for (i, file_data) in files.into_iter().enumerate() {
println!("Processing file {}", i + 1);
// 트랜잭션 생성 및 서명
let transaction = signer.create_transaction(file_data, Some(anchor))?;
let signed_transaction = signer.sign_transaction(transaction)?;
// 트랜잭션 제출
client
.post_transaction(node_addr, signed_transaction.header.clone())
```

```
.await?;
// 청크 업로드
client
.upload_chunks(node_addr, &signed_transaction)
.await?;
tx_ids.push(signed_transaction.header.id);
// 요청 간 간격 (선택사항)
tokio::time::sleep(std::time::Duration::from_millis(100)).await;
}
Ok(tx_ids)
}
```

2. 스트리밍 업로드

```
use futures::stream::{self, StreamExt};

async fn streaming_upload(
client: &IrysApiClient,
node_addr: SocketAddr,
signer: &IrysSigner,
data_stream: impl Iterator<Item = eyre::Result<Vec<u8>>> + Send,
) -> eyre::Result<()> {
// 앵커 조회
```

```
let anchor = client.get_anchor(node_addr).await?;

// 트랜잭션 생성 (데이터 저장하지 않음)

let transaction = signer.create_transaction_from_iter(

data_stream,

Some(anchor),

false, // 데이터를 메모리에 저장하지 않음

)?;

// 트랜잭션 서명

let signed_transaction = signer.sign_transaction(transaction)?;

// 트랜잭션 제출

client

.post_transaction(node_addr, signed_transaction.header.clone())

.await?;

// 스트리밍 방식으로 청크 업로드

// Note: 실제 데이터 스트림을 다시 생성해야 함

// client.upload_chunks_iter(node_addr, &signed_transaction, data_stream).await?

Ok(())

}
```

3. 오류 처리 및 재시도

```
use tokio::time::{sleep, Duration};
```

```
async fn robust_upload(
client: &IrysApiClient,
node_addr: SocketAddr,
signer: &IrysSigner,
data: Vec<u8>,
max_retries: u32,
) -> eyre::Result<H256> {
let mut retries = 0;
loop {
match attempt_upload(client, node_addr, signer, &data).await {
Ok(tx_id) => return Ok(tx_id),
Err(e) if retries < max_retries => {
retries += 1;
println!("Upload failed (attempt {}): {}. Retrying...", retries, e);
sleep(Duration::from_secs(2_u64.pow(retries))).await; // 지수 백오프
}
Err(e) => return Err(e),
}
}
}


async fn attempt_upload(
client: &IrysApiClient,
```

```
node_addr: SocketAddr,
signer: &IrysSigner,
data: &[u8],
) -> eyre::Result<H256> {
let anchor = client.get_anchor(node_addr).await?;
let transaction = signer.create_transaction(data.to_vec(), Some(anchor))?;
let signed_transaction = signer.sign_transaction(transaction)?;
client
.post_transaction(node_addr, signed_transaction.header.clone())
.await?;
client
.upload_chunks(node_addr, &signed_transaction)
.await?;
Ok(signed_transaction.header.id)
}
```

노드 정보 및 모니터링

1. 노드 상태 확인

```
async fn check_node_status(
client: &IrysApiClient,
node_addr: SocketAddr,
```

```
) -> eyre::Result<()> {
let node_info = client.node_info(node_addr).await?;
println!("Node Information:");
println!(" Version: {}", node_info.version);
println!(" Chain ID: {}", node_info.chain_id);
println!(" Height: {}", node_info.height);
println!(" Peer count: {}", node_info.peer_count);
println!(" Is syncing: {}", node_info.is_syncing);
if node_info.is_syncing {
println!(" Sync target: {}", node_info.current_sync_height);
}
Ok(())
}
```

2. 블록 인덱스 조회

```
use irys_types::BlockIndexQuery;

async fn query_block_index(
client: &IrysApiClient,
node_addr: SocketAddr,
start_height: u64,
limit: u64,
```

```
) -> eyre::Result<()> {
let query = BlockIndexQuery {
height: start_height,
limit,
};
let block_items = client.get_block_index(node_addr, query).await?;
println!("Block index (from height {}):", start_height);
for item in block_items {
println!(" Height: {}, Hash: {:?}", item.height, item.block_hash);
}
Ok(())
}
```

성능 최적화

1. 연결 풀링

```
use std::sync::Arc;

// 여러 작업에서 클라이언트 재사용
let client = Arc::new(IrysApiClient::new());

// 병렬 업로드
```

```
let handles: Vec<_> = files
.into_iter()
.map(|file_data| {
let client = Arc::clone(&client);
let signer = signer.clone();
tokio::spawn(async move {
upload_single_file(client.as_ref(), node_addr, &signer, file_data).await
})
})
.collect();


// 모든 업로드 완료 대기
let results: Vec<_> = futures::future::join_all(handles).await;
```

2. 청크 크기 최적화

```
// 큰 파일의 경우 청크 크기 조정
let optimized_signer = IrysSigner {
signer: signer.signer.clone(),
chain_id: signer.chain_id,
chunk_size: 512 * 1024, // 512KB 청크
};
```

3. 메모리 효율적인 처리

```
// 대용량 파일을 위한 스트리밍 처리
fn create_file_stream(file_path: &str, chunk_size: usize) -> impl Iterator<Item
let file = std::fs::File::open(file_path).unwrap();
let mut reader = std::io::BufReader::new(file);
std::iter::from_fn(move || {
let mut buffer = vec![0; chunk_size];
match reader.read(&mut buffer) {
Ok(0) => None, // EOF
Ok(n) => {
buffer.truncate(n);
Some(Ok(buffer))
}
Err(e) => Some(Err(e.into())),
}
})
}
```

테스팅

1. 단위 테스트

```rust
#[cfg(test)]
mod tests {
use super::*;
#[tokio::test]
async fn test_transaction_creation() {
let config = ConsensusConfig::testing();
let signer = IrysSigner::random_signer(&config);
let data = b"Hello, Irys!".to_vec();
let transaction = signer.create_transaction(data, None).unwrap();
let signed_transaction = signer.sign_transaction(transaction).unwrap();
assert!(signed_transaction.header.is_signature_valid());
assert_eq!(signed_transaction.header.data_size, 13);
}
#[tokio::test]
async fn test_commitment_transaction() {
let config = ConsensusConfig::testing();
let signer = IrysSigner::random_signer(&config);
let anchor = H256::random();
let stake_tx = CommitmentTransaction::new_stake(&config, anchor);
let signed_stake_tx = signer.sign_commitment(stake_tx).unwrap();
assert!(signed_stake_tx.is_signature_valid());
assert_eq!(signed_stake_tx.commitment_type, CommitmentType::Stake);
```

```
}

}
```

2. 통합 테스트

```
#[cfg(test)]

mod integration_tests {

use  super::*;

#[tokio::test]

#[ignore] // 실제 노드가 필요한 테스트

async  fn  test_full_upload_workflow() {

let  client = IrysApiClient::new();

let  node_addr: SocketAddr = "127.0.0.1:8081".parse().unwrap();

let  config = ConsensusConfig::testing();

let  signer = IrysSigner::random_signer(&config);

let  test_data = b"Integration test data".to_vec();

// 전체 워크플로우 테스트

let  result = upload_file_workflow(&client, node_addr, &signer, test_data).await

assert!(result.is_ok());

}

}
```

오류 처리

일반적인 오류 시나리오

```
use irys_api_client::StatusCode;


async fn handle_api_errors(
client: &IrysApiClient,
node_addr: SocketAddr,
tx_header: DataTransactionHeader,
) -> eyre::Result<()> {
match client.post_transaction(node_addr, tx_header).await {
Ok(_) => println!("Transaction submitted successfully"),
Err(e) => {
if let Some(reqwest_error) = e.downcast_ref::<reqwest::Error>() {
if let Some(status) = reqwest_error.status() {
match status {
StatusCode::BAD_REQUEST => {
println!("Invalid transaction data: {}", e);
}
StatusCode::PAYMENT_REQUIRED => {
println!("Insufficient funds: {}", e);
}
StatusCode::INTERNAL_SERVER_ERROR => {
println!("Server error, retry later: {}", e);
```

```
}
_ => {
println!("Unexpected error: {}", e);
}
}
}
}
return Err(e);
}
}
Ok(())
}
```

모범 사례

1. 보안

- 개인키를 안전하게 저장하고 관리
- 프로덕션 환경에서는 하드웨어 보안 모듈(HSM) 사용 고려
- 트랜잭션 서명 전 항상 내용 검증

2. 성능

- 대용량 파일은 스트리밍 방식으로 처리
- 배치 작업 시 적절한 간격으로 요청 분산

- 연결 풀링 및 재사용으로 네트워크 오버헤드 최소화

3. 안정성

- 네트워크 오류에 대한 재시도 로직 구현
- 트랜잭션 상태 추적 및 모니터링
- 적절한 타임아웃 설정

4. 모니터링

- 트랜잭션 처리 시간 측정
- 오류율 및 성공률 추적
- 노드 상태 정기적 확인

이 가이드를 통해 Irys 체인 클라이언트 SDK의 모든 기능을 효과적으로 활용할 수 있습니다. 추가적인 질문이나 특정 사용 사례에 대한 도움이 필요하시면 언제든지 문의해 주세요.